Reporte

Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Implementación del núcleo de negocio TechStore
Fecha: 28/10/2026	

Bibliografía (Formato APA)

Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley Professional.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley Professional.

Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Millett, S., & Tune, N. (2015). *Patterns, principles, and practices of domain-driven design*. Wrox.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley Professional.

Equipo 6

Jesús Eduardo Moreno Osoria	**2136063**
Osvaldo Salvador Cárdenas Mejorado	**2141553**

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

Domain Events y Diagrama de Estados

TechStore – Ciclo de vida del pedido

1. ¿Qué es un Domain Event?

Un Domain Event es el registro de algo importante que ya ocurrió en el negocio. Por eso su nombre está en pasado (OrderConfirmed = «pedido confirmado»), no se puede modificar después de crearse y guarda la fecha y hora en que sucedió. En TechStore cada cambio de estado del pedido genera exactamente un evento.

2. Diagrama de estados

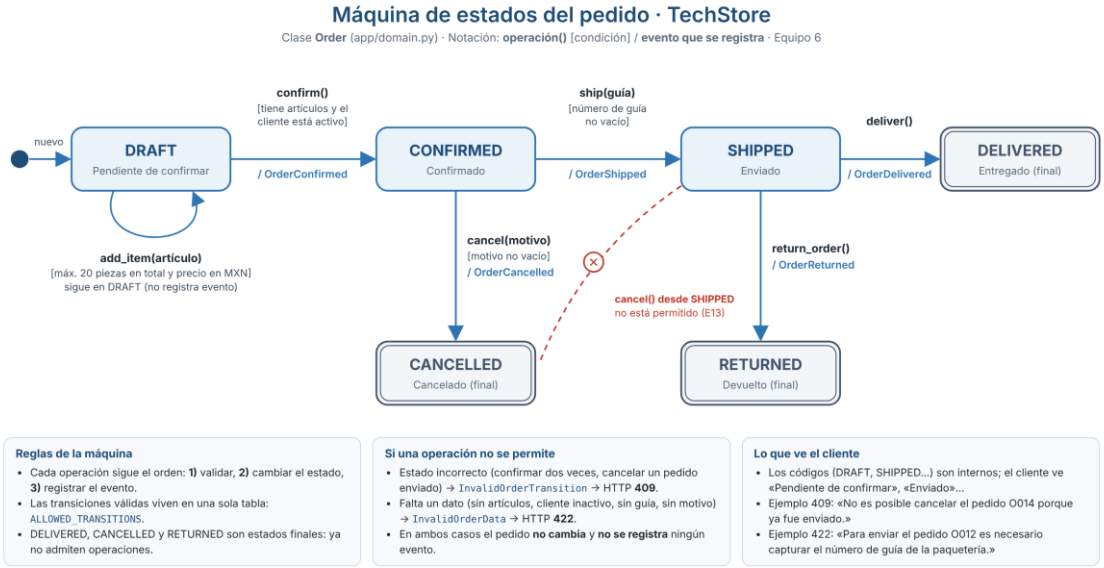


Figura 1. Máquina de estados del pedido TechStore.

El pedido nace en DRAFT («Pendiente de confirmar») y solo puede avanzar por las flechas del diagrama. DELIVERED, CANCELLED y RETURNED son estados finales. `add_item()` no es una transición: solo funciona en DRAFT y no registra eventos.

3. Transiciones permitidas y precondiciones

Transición y método	Precondiciones (en este orden)	Si falla	Evento
DRAFT → CONFIRMED `confirm()`	1) estado DRAFT 2) al menos un artículo 3) cliente activo	1) InvalidOrderTransition (409) 2–3) InvalidOrderData (422)	OrderConfirmed
CONFIRMED → SHIPPED `ship(guía)`	1) estado CONFIRMED 2) número de guía no vacío	1) InvalidOrderTransition (409) 2) InvalidOrderData (422)	OrderShipped
CONFIRMED → CANCELLED `cancel(motivo)`	1) estado CONFIRMED 2) motivo no vacío	1) InvalidOrderTransition (409) 2) InvalidOrderData (422)	OrderCancelled
SHIPPED → DELIVERED `deliver()`	estado SHIPPED	InvalidOrderTransition (409)	OrderDelivered
SHIPPED → RETURNED `return_order()`	estado SHIPPED	InvalidOrderTransition (409)	OrderReturned

Ejemplos de saltos prohibidos: enviar sin confirmar (DRAFT → SHIPPED), confirmar dos veces (E12), cancelar un pedido enviado (E13), entregar sin enviar (CONFIRMED → DELIVERED) y cualquier operación desde un estado final. La prueba `test_toda_transicion_fuera_del_diagrama_se_rechaza_sin_efectos` recorre los 25 saltos prohibidos y comprueba que ninguno cambia el pedido.

4. Estructura de un evento

Fragmento real de `app/domain.py`. La clase base guarda el pedido y la fecha; cada evento agrega solo los datos que le corresponden:

```
# frozen=True: una vez creado, el evento no se puede modificar
@dataclass(frozen=True, kw_only=True)
class DomainEvent:
    order_id: str                                          # pedido
    occurred_at: datetime = field(default_factory=_now)  # fecha UTC


    @property
    def name(self) -> str:
        return type(self).__name__   # "OrderConfirmed", ...

@dataclass(frozen=True, kw_only=True)
class OrderConfirmed(DomainEvent):
    customer_id: str
    total: Money
```

5. Los cinco eventos

Evento	Cuándo se registra	Datos propios	Aviso que genera EmailNotifier
OrderConfirmed	DRAFT → CONFIRMED	customer_id, total	«Hola Ana Torres: confirmamos tu pedido O001 por $2,100.00 MXN. ¡Gracias por tu compra!»
OrderShipped	CONFIRMED → SHIPPED	tracking_number	«…tu pedido O001 ya va en camino. Número de guía: TRK-0001.»
OrderCancelled	CONFIRMED → CANCELLED	reason	«…tu pedido O002 fue cancelado. Motivo: …»
OrderDelivered	SHIPPED → DELIVERED	—	«…tu pedido O016 fue entregado. ¡Que lo disfrutes!»
OrderReturned	SHIPPED → RETURNED	reason (opcional)	«…recibimos la devolución de tu pedido O015.»

Todos los eventos incluyen además `order_id` y `occurred_at`.

6. Cómo se registran y se publican

Dentro de `Order`, cada método sigue siempre los mismos tres pasos:

```
def confirm(self) -> None:
    # 1) validar: ¿existe la flecha?, ¿tiene artículos?, ¿cliente activo?
    self._ensure_can_move_to(OrderStatus.CONFIRMED, "confirmar")
    if not self._items: raise InvalidOrderData(...)
    if not self._customer.active: raise InvalidOrderData(...)
    # 2) cambiar el estado
    self._status = OrderStatus.CONFIRMED
    # 3) registrar el evento
    self._record(OrderConfirmed(order_id=..., customer_id=..., total=...))
```

Después, `OrderService` (el Application Service) guarda el pedido y publica solo los eventos nuevos en el `EventBus`. El bus los entrega a los suscriptores que se registraron para ese tipo de evento:

```
def _execute(self, order_id, operation):
    order = self.get(order_id)
    already_published = len(order.events)
    # si el dominio rechaza, la excepción sale aquí y no se publica nada
    operation(order)
    self.repository.save(order)
    self.bus.publish(order.events[already_published:])
    return order
```

Pieza	Responsabilidad
Order	Decide si la operación es válida y registra el evento. No conoce a los suscriptores.
OrderService	Guarda el pedido y publica los eventos nuevos después de una operación exitosa.

Pieza	Responsabilidad
EventBus	Entrega cada evento a sus suscriptores. Si un suscriptor falla, anota el error y sigue: el hecho ya ocurrió y no se deshace.
EmailNotifier	Suscriptor de ejemplo: arma el aviso para el cliente con su nombre (correo simulado).

Gracias a esta separación se podrían agregar otros suscriptores (inventario, facturación) sin tocar `Order`.

7. ¿Por qué solo después de una transición válida?

Porque un evento afirma que algo pasó. Si se registrara antes de validar y luego la validación fallara, el sistema anunciaría un hecho falso: el cliente recibiría «tu pedido va en camino» de un pedido que nunca se envió. Los escenarios lo confirman:

Escenario	Qué se intentó	Eventos que quedaron
E11 (O012)	Confirmar y después enviar sin guía	Solo OrderConfirmed: el envío falló y no hay OrderShipped
E12 (O013)	Confirmar dos veces	Un solo OrderConfirmed: el segundo intento no duplica el evento
E13 (O014)	Confirmar, enviar y cancelar	OrderConfirmed y OrderShipped: no hay OrderCancelled

8. Cómo se ven en la API

Los eventos de un pedido aparecen en `GET /orders/{id}` con un texto para el cliente (`description`). Ejemplo real de O001 después de confirmarlo y enviarlo:

```
"status": "SHIPPED",
"status_label": "Enviado",
"events": [
  { "name": "OrderConfirmed", "description": "Pedido confirmado",
    "order_id": "O001",
    "occurred_at": "2026-10-28T00:21:56.538157Z",
    "data": { "customer_id": "C001", "total": "2100.00 MXN" } },
  { "name": "OrderShipped", "description": "Pedido enviado",
    "order_id": "O001",
    "occurred_at": "2026-10-28T00:21:56.543820Z",
    "data": { "tracking_number": "TRK-0001" } }
]
```

Además, `GET /events` lista todos los eventos publicados y `GET /notifications` muestra los avisos que generó el EmailNotifier.